

BLOQUE II: RESOLUCIÓN DE SISTEMAS DE ECUACIONES.

II.3 – Sistemas no lineales: Método de Newton-Raphson.

→ Sistemas no lineales.

Método de sustituciones sucesivas.

Método de Newton-Raphson.

Otras aproximaciones: métodos de minimización.

→ Los sistemas de ecuaciones no lineales sólo pueden resolverse por métodos iterativos.

MÉTODO DE SUSTITUCIONES SUCESIVAS.

→ Generalización del método ya visto para hallar los ceros de una ecuación no lineal (iteración de punto fijo, $x=g(x)$).

→ Dado el sistema de ecuaciones:

$$f_1(x_1, x_2, \dots, x_n) = 0$$

$$f_2(x_1, x_2, \dots, x_n) = 0$$

$$\vdots$$

$$f_n(x_1, x_2, \dots, x_n) = 0$$

siempre se puede reordenar de la forma equivalente:

$$x_1 = F_1(\underline{x_1}, x_2, \dots, x_n)$$

$$x_2 = F_2(x_1, \underline{x_2}, \dots, x_n) \quad \rightarrow \quad x_i = F_i(x_1, x_2, \dots, x_n); i=1, 2, \dots, n$$

$$\vdots$$

$$x_n = F_n(x_1, x_2, \dots, \underline{x_n})$$

→ Reordenamiento similar al llevado a cabo en los métodos iterativos de Jacobi y Gauss-Seidel.

→ La condición iterativa: $x_i^{(k+1)} = F_i(x_1^{(k)}, \dots, x_n^{(k)}); i=1, 2, \dots, n$, es análoga a la derivada para el procedimiento de Jacobi (*desplazamientos simultáneos*).

→ Si se modifica como: $x_i^{(k+1)} = F_i(x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}, x_i^{(k)}, \dots, x_n^{(k)}); i=1, 2, \dots, n$, se hace análoga a la utilizada en el procedimiento de Gauss-Seidel (*desplazamientos sucesivos*).

→ Si $F_i(x_1, \dots, x_n)$ y sus derivadas parciales son continuas en un intervalo alrededor de una raíz $(r_1, r_2, \dots, r_n)$ del sistema de ecuaciones $x_i = F_i(x_1, x_2, \dots, x_n)$, y si se cumple que

$$\sum_{j=1}^n \left| \frac{\partial F_i(x_1, \dots, x_n)}{\partial x_j} \right| \leq 1 \quad (i=1, 2, \dots, n) \text{ para todo } x_i \text{ en dicho intervalo, entonces la forma iterativa}$$

$x_i^{(k+1)} = F_i(x_1^{(k)}, \dots, x_n^{(k)}); i=1, 2, \dots, n$, converge a una raíz $(r_1, \dots, r_n)$, si la aproximación inicial $(x_1^{(1)}, x_2^{(1)}, \dots, x_n^{(1)})$ se toma en dicho intervalo.

→ Es una condición suficiente pero no necesaria. ←

Ejemplo:

$$\begin{aligned} f(x, y) &= \frac{1}{2} \operatorname{sen}(x \cdot y) - \frac{y}{4\pi} - \frac{x}{2} = 0 & x &= \operatorname{sen}(x \cdot y) - \frac{y}{2\pi} \Rightarrow x = F_1(x, y) \\ g(x, y) &= \left(1 - \frac{1}{4\pi}\right)(e^{2x} - e) + \frac{e \cdot y}{\pi} - 2e \cdot x = 0 & y &= 2\pi \cdot x - \left(\pi - \frac{1}{4}\right)(e^{2x-1} - 1) \Rightarrow y = F_2(x, y) \end{aligned} \Rightarrow$$

→ Valores iniciales: $(x^{(1)}, y^{(1)}) = (0.4, 3.0)$; Solución real $(0.5, \pi)$.

(a) Con la **ecuación iterativa de tipo Jacobi** se obtiene la siguiente sucesión:

(k)	$x^{(k)}$	$y^{(k)}$
1	0.4	3.0
2	0.455	3.03
3	0.499	3.11
4	0.505	3.14
5	0.500	3.14
6	0.500	π

$$\begin{aligned} x^{(k+1)} &= F_1(x^{(k)}, y^{(k)}) \\ y^{(k+1)} &= F_2(x^{(k)}, y^{(k)}) \end{aligned} \leftarrow$$

(b) Con la **ecuación iterativa de tipo Gauss-Seidel** se obtiene:

(k)	$x^{(k)}$	$y^{(k)}$
1	0.4	3.0
2	0.455	3.11
3	0.493	3.14
4	0.500	π
5	0.500	π

$$\begin{aligned} \leftarrow x^{(k+1)} &= F_1(x^{(k)}, y^{(k)}) \\ y^{(k+1)} &= F_2(x^{(k+1)}, y^{(k)}) \end{aligned}$$

→ En este segundo caso la convergencia es más rápida.

→ En contraste con los métodos directos, no se acumulan los errores de redondeo de una iteración a otra.

→ Después de cada iteración (k) , el nuevo resultado es tomado como una nueva estimación de la aproximación inicial a la raíz real.

Ejemplo:

$$4x + y \sin(z) - 1 = 0$$

$$\sqrt{x} + \ln(y) + \frac{1}{2} \cos(z) = 0 \rightarrow \text{reordenando} \rightarrow y = e^{[-\cos(z)/2 - \sqrt{x}]} \rightarrow y = F_2(x, y, z)$$

$$y \ln(4x) - 2z - e^z + 1 = 0 \rightarrow z = \frac{1}{2} [1 + y \ln(4x) - e^z] \rightarrow z = F_3(x, y, z)$$

→ Valores iniciales: $(x^{(1)}, y^{(1)}, z^{(1)}) = (1/3, 1/3, \pi/6)$; Solución real $(1/4, 1/e, 0)$.

→ Con la **ecuación iterativa de tipo Gauss-Seidel** se obtiene:

(k)	$x^{(k)}$	$y^{(k)}$	$z^{(k)}$
1	1/3	1/3	$\pi/6$
2	0.208333	0.410882	-0.381502
3	0.288244	0.367535	0.184741
4	0.233121	0.377449	-0.114646
5	0.260795	0.365168	0.061877
6	0.250000	0.367879	0.000000

$$\begin{aligned} x^{(k+1)} &= F_1(x^{(k)}, y^{(k)}, z^{(k)}) \\ y^{(k+1)} &= F_2(x^{(k+1)}, y^{(k)}, z^{(k)}) \\ z^{(k+1)} &= F_3(x^{(k+1)}, y^{(k+1)}, z^{(k)}) \end{aligned}$$

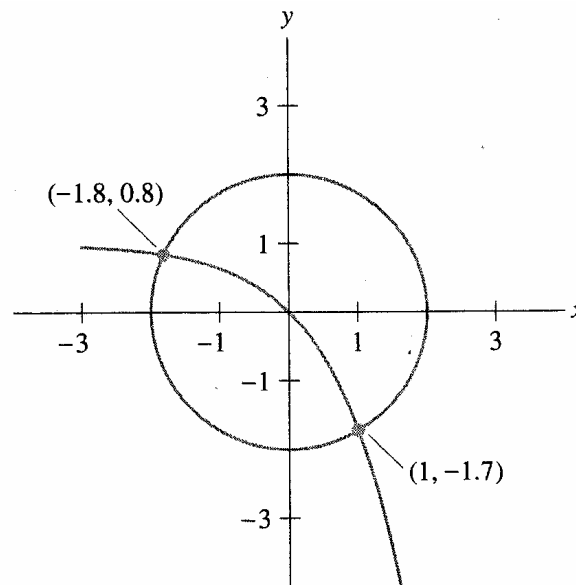
MÉTODO DE NEWTON-RAPHSON.

→ Consideremos un sistema de ecuaciones no lineales:

$$x^2 + y^2 = 4 \rightarrow f(x, y) = x^2 + y^2 - 4 = 0$$

$$e^x + y = 1 \rightarrow g(x, y) = e^x + y - 1 = 0$$

Gráficamente, la solución a dicho sistema viene dada por las intersecciones de las dos curvas.



→ Extensión del método de Newton a un sistema de ecuaciones (no lineales).

→ Sea $(x=\mathbf{r}, y=\mathbf{s})$ una raíz del sistema de ecuaciones.

→ Desarrollo de $f(x,y)$ y $g(x,y)$ en serie de Taylor alrededor de un punto inicial (x_i, y_i) cerca de

$$f(r,s) = 0 = f(x_i, y_i) + \underbrace{\frac{\partial f(x_i, y_i)}{\partial x}}_{f_x(x_i, y_i)}(r - x_i) + \underbrace{\frac{\partial f(x_i, y_i)}{\partial y}}_{f_y(x_i, y_i)}(s - y_i) + \dots$$

la raíz:

$$g(r,s) = 0 = g(x_i, y_i) + \underbrace{\frac{\partial g(x_i, y_i)}{\partial x}}_{g_x(x_i, y_i)}(r - x_i) + \underbrace{\frac{\partial g(x_i, y_i)}{\partial y}}_{g_y(x_i, y_i)}(s - y_i) + \dots$$

→ Truncamos la serie y nos quedamos con las aproximaciones lineales de $f(x,y)$ y $g(x,y)$. En notación matricial:

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} f(x_i, y_i) \\ g(x_i, y_i) \end{bmatrix} + \begin{bmatrix} f_x(x_i, y_i) & f_y(x_i, y_i) \\ g_x(x_i, y_i) & g_y(x_i, y_i) \end{bmatrix} \begin{bmatrix} (r - x_i) \\ (s - y_i) \end{bmatrix}$$

→ Haciendo $\Delta x_i = r - x_i$ y $\Delta y_i = s - y_i$, y reordenando obtenemos el sistema de ecuaciones:

$$\begin{bmatrix} f_x(x_i, y_i) & f_y(x_i, y_i) \\ g_x(x_i, y_i) & g_y(x_i, y_i) \end{bmatrix} \begin{bmatrix} \Delta x_i \\ \Delta y_i \end{bmatrix} = - \begin{bmatrix} f(x_i, y_i) \\ g(x_i, y_i) \end{bmatrix}$$

→ A la matriz de coeficientes del sistema se la llama matriz jacobiana:

$$J = \begin{bmatrix} f_x(x_i, y_i) & f_y(x_i, y_i) \\ g_x(x_i, y_i) & g_y(x_i, y_i) \end{bmatrix}$$

→ Resolviendo este sistema (por el método LU o eliminación gaussiana) obtendremos estimaciones mejoradas de las raíces (r, s) dadas por: $x_{i+1} = x_i + \Delta x_i$; $y_{i+1} = y_i + \Delta y_i$

→ Para un sistema de 2 ecuaciones, el determinante de la matriz de coeficientes es: $D = f_x(x_i, y_i) \cdot g_y(x_i, y_i) - f_y(x_i, y_i) \cdot g_x(x_i, y_i)$, de forma que los incrementos óptimos buscados

son:

$$\Delta x_i = \left(\frac{g(x_i, y_i) \cdot f_y(x_i, y_i) - f(x_i, y_i) \cdot g_y(x_i, y_i)}{D} \right)$$
$$\Delta y_i = \left(\frac{f(x_i, y_i) \cdot g_x(x_i, y_i) - g(x_i, y_i) \cdot f_x(x_i, y_i)}{D} \right)$$

→ El proceso se repite de forma iterativa, reemplazando i por $i+1$ en el sistema de ecuaciones, hasta que después de las sucesivas aproximaciones a las raíces, $f \approx 0$ y $g \approx 0$.

→ **Observaciones:**

→ El método reduce la solución de un sistema de ecuaciones no lineales, a la resolución de un sistema de ecuaciones lineales. Pero no determina directamente las incógnitas (x, y) , sino los incrementos óptimos $(\Delta x_i, \Delta y_i)$ para mejorar sucesivamente las aproximaciones a las raíces verdaderas (r, s) .

→ Convergencia cuadrática si se está cerca de las raíces. (si $D \neq 0$)

→ Evaluaciones funcionales: $\sim (n^2 + n)$ en cada iteración.

Ejemplo:

$$f(x, y) = \frac{1}{2} \operatorname{sen}(x \cdot y) - \frac{y}{4\pi} - \frac{x}{2} = 0$$
$$g(x, y) = \left(1 - \frac{1}{4\pi}\right)(e^{2x} - e) + \frac{e \cdot y}{\pi} - 2e \cdot x = 0$$

las derivadas parciales son:

$$f_x = -\frac{1}{2} + \frac{y \cdot \cos(x \cdot y)}{2} \quad f_y = -\frac{1}{4\pi} + \frac{x \cdot \cos(x \cdot y)}{2}$$
$$g_x = -2e + \left(2 - \frac{1}{2\pi}\right)e^{2x} \quad g_y = \frac{e}{\pi}$$

lo que da el siguiente sistema de ecuaciones:

$$\begin{bmatrix} -\frac{1}{2} + \frac{y_i \cdot \cos(x_i \cdot y_i)}{2} & -\frac{1}{4\pi} + \frac{x_i \cdot \cos(x_i \cdot y_i)}{2} \\ -2e + \left(2 - \frac{1}{2\pi}\right)e^{2x_i} & \frac{e}{\pi} \end{bmatrix} \cdot \begin{bmatrix} \Delta x_i \\ \Delta y_i \end{bmatrix} = - \begin{bmatrix} \frac{1}{2} \operatorname{sen}(x_i \cdot y_i) - \frac{y_i}{4\pi} - \frac{x_i}{2} \\ \left(1 - \frac{1}{4\pi}\right)(e^{2x_i} - e) + \frac{e \cdot y_i}{\pi} - 2e \cdot x_i \end{bmatrix}$$

$$x_{i+1} = x_i + \Delta x_i \quad ; \quad y_{i+1} = y_i + \Delta y_i$$

→ Dato inicial (1): tomamos como primera aproximación a (r, s) $(x_0, y_0)=(0.4, 3.0)$

i	x	y	f	g	f_x	f_y	g_x	g_y	D	Δx_i	Δy_i
0	0.400	3.000	0.0272	-0.0324	0.0435	-0.0071	-1.34	0.865	0.0281	-0.831	-1.249
1	-0.431	1.751	-0.266	1.74	0.138	-0.236	-4.66	0.865	-0.982	0.186	-1.018
2	-0.245	0.733	-0.0251	0.0303	-0.139	-0.200	-4.31	0.865	-0.984	-0.016	-0.114
3	-0.261	0.619	0.0009	0.0003	-0.195	-0.208	-4.35	0.865	-1.07	0.0007	0.003
4	-0.260	0.622	0.0000	0.0000	-0.193	-0.208	-4.34	0.865	-1.07	0.0000	0.0000
5	-0.260	0.622	0.0000	0.0000						↑	↑

Conduce a la solución $(r,s)=(-0.260,0.622)$

→ Dato inicial (2): tomamos como primera aproximación a (r, s) $(x_0, y_0)=(0.6, 3.0)$

i	x	y	f	g	f_x	f_y	g_x	g_y	D	Δx_i	Δy_i
0	0.600	3.000	-0.0518	-0.112	-0.841	-0.148	0.675	0.865	-0.627	-0.098	-0.206
1	0.502	3.206	-0.0066	0.0549	-0.563	-0.0894	-0.411	0.865	-0.524	-0.001	-0.064
2	0.501	3.142	-0.0003	0.0000	-0.503	-0.0801	-0.426	0.865	-0.470	-0.0006	-0.0004
3	0.500	3.142	-0.0000	-0.0000	-0.500	-0.0796	-0.433	0.865	-0.467	-0.0000	-0.0000
4	0.500	3.142	-0.0000	0.0000						↑	↑

Conduce a la solución alternativa $(r,s)=(0.5,\pi)$

Ejemplo:

$$3x - \cos(yz) - \frac{1}{2} = 0$$

$$x^2 - 81(y + 0.1)^2 + \sin(z) + 1.06 = 0 \quad ; \text{ tiene una solución aproximada en } (0.5, 0, -0.52359877)$$

$$e^{-xy} + 20z + \frac{10\pi - 3}{3} = 0$$

lo que da el siguiente sistema de ecuaciones:

$$\begin{bmatrix} 3 & z_i \sin(y_i z_i) & y_i \sin(y_i z_i) \\ 2x_i & -162(y_i + 0.1) & \cos(z_i) \\ -y_i e^{-x_i y_i} & -x_i e^{-x_i y_i} & 20 \end{bmatrix} \cdot \begin{bmatrix} \Delta x_i \\ \Delta y_i \\ \Delta z_i \end{bmatrix} = - \begin{bmatrix} 3x_i - \cos(y_i z_i) - \frac{1}{2} \\ x_i^2 - 81(y_i + 0.1)^2 + \sin(z_i) + 1.06 \\ e^{-x_i y_i} + 20z_i + \frac{10\pi - 3}{3} \end{bmatrix}$$

$$x_{i+1} = x_i + \Delta x_i \quad ; \quad y_{i+1} = y_i + \Delta y_i \quad ; \quad z_{i+1} = z_i + \Delta z_i$$

→ Tomando como primera aproximación a la raíz: $(x_0, y_0, z_0) = (0.1, 0.1, -0.1)$

(i)	x	y	z	$\ \mathbf{x}^{(i+1)} - \mathbf{x}^{(i)}\ $
0	0.10000000	0.10000000	-0.10000000	
1	0.50003702	0.01946686	-0.52152047	0.422
2	0.50004593	0.00158859	-0.52355711	$1.79 \cdot 10^{-2}$
3	0.50000034	0.00001244	-0.52359845	$1.58 \cdot 10^{-3}$
4	0.50000000	0.00000000	-0.52359877	$1.24 \cdot 10^{-5}$
5	0.50000000	0.00000000	-0.52359877	0

Tiene una solución aproximada en (0.5, 0, -0.52359877)

→ Generalización para un sistema de n ecuaciones:

→ Definamos: $f_{ij}(\vec{x}^{(k)}) = \frac{\partial f_i(\vec{x}^{(k)})}{\partial x_j}$, donde $(\vec{x}^{(k)}) = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})$ es la aproximación actual (k) para la solución del sistema de ecuaciones.

→ El sistema de ecuaciones con incógnitas $(\Delta \vec{x}^{(k)})$ tomará la forma:

$$\underbrace{\begin{bmatrix} f_{11} & f_{12} & \dots & f_{1n} \\ f_{21} & f_{22} & \dots & f_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ f_{n1} & f_{n2} & \dots & f_{nn} \end{bmatrix}}_{\text{Jacobiano}} \begin{bmatrix} \Delta x_1^{(k)} \\ \Delta x_2^{(k)} \\ \vdots \\ \Delta x_n^{(k)} \end{bmatrix} = - \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{bmatrix}$$

donde las derivadas parciales $(f_{11}, \dots, f_{nn})$ y las funciones $(f_1, \dots, f_n)$ están evaluadas en el valor actual de la aproximación a la raíz $(\vec{x}^{(k)})$.

→ Resolviendo el sistema obtendremos una estimación mejorada de la raíz $(\vec{r})$ dada por $\vec{x}^{(k+1)} = \vec{x}^{(k)} + \Delta \vec{x}^{(k)}$, que se volverá a utilizar reemplazando k por $k+1$ en el sistema de ecuaciones, hasta que después de las sucesivas aproximaciones, $f_1, \dots, f_n \approx 0$.

→ **Inconveniente**: dificultad de introducir todas las derivadas parciales de cada una de las funciones.

→ **Alternativa**: aproximar las derivadas parciales por diferencias finitas.

→ Para una función de una variable se puede aproximar la derivada mediante:

- Fórmula de diferencias progresivas para $f'(x)$ de dos puntos:

$$\Rightarrow \boxed{f'(x) = \frac{f(x+h) - f(x)}{h}} \Leftarrow \text{(con } h \text{ suficientemente pequeño)}$$

- Fórmula de diferencias centrales para $f'(x)$ de tres puntos:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

- Fórmula de diferencias progresivas para $f'(x)$ de tres puntos:

$$f'(x) = \frac{-f(x+2h) + 4f(x+h) - 3f(x)}{2h}$$

- Fórmula de diferencias progresivas para $f'(x)$ de cinco puntos:

$$f'(x) = \frac{-3f(x+4h) + 16f(x+3h) - 36f(x+2h) + 48f(x+h) - 25f(x)}{12h}$$

- Fórmula de diferencias centrales para $f'(x)$ de cinco puntos:

$$f'(x) = \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}$$

→ Por extensión, definimos la derivada parcial de una función de varias variables:

$$f_{11} \approx \frac{f_1(x_1 + h, x_2, \dots, x_n) - f_1(x_1, x_2, \dots, x_n)}{h}$$

$$f_{12} \approx \frac{f_1(x_1, x_2 + h, \dots, x_n) - f_1(x_1, x_2, \dots, x_n)}{h}$$

$$\dots\dots\dots$$
$$f_{1n} \approx \frac{f_1(x_1, x_2, \dots, x_n + h) - f_1(x_1, x_2, \dots, x_n)}{h}$$

→ En notación más compacta: $f_{ij}(\vec{x}^{(k)}) = \frac{\partial f_i(\vec{x}^{(k)})}{\partial x_j} \approx \frac{f_i(\vec{x}^{(k)} + \vec{e}_j h) - f_i(\vec{x}^{(k)})}{h}$, donde

$(\vec{x}^{(k)}) = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})$, y $\vec{e}_j$ es un vector cuyas componentes son todas cero excepto la j -ésima, que es 1.

→ **Otras aproximaciones - métodos de minimización:**

→ Podemos transformar el problema de encontrar la solución de un sistema de ecuaciones no lineales en uno de minimización.

→ El problema de encontrar las raíces del sistema de ecuaciones no lineales:

$$f_1(x_1, x_2, \dots, x_n) = 0$$

$$f_1(x_1, x_2, \dots, x_n) = 0$$

.....

$$f_n(x_1, x_2, \dots, x_n) = 0$$

es equivalente a minimizar la función escalar:

$$S(x_1, x_2, \dots, x_n) = f_1^2(x_1, x_2, \dots, x_n) + \dots + f_n^2(x_1, x_2, \dots, x_n) = \sum_{i=1}^n [f_i(x_1, x_2, \dots, x_n)]^2$$

lo que permite utilizar algoritmos de minimización para encontrar los ceros de un sistema de ecuaciones no lineales.

- Método del descenso más rápido o de la máxima pendiente:

→ Se busca el mínimo de la función $S(x_1, \dots, x_n)$, moviéndonos en la dirección negativa de su gradiente (dirección de máxima disminución de S), de modo que definimos el siguiente proceso

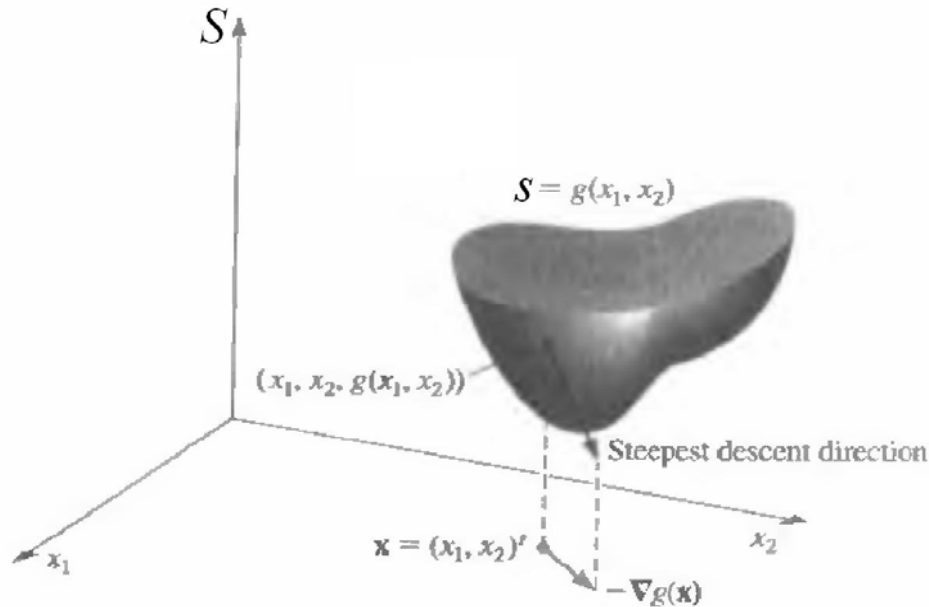
iterativo:

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} - \alpha \nabla S(\vec{x}^{(k)})$$

donde el gradiente es: $\nabla S(\vec{x}^{(k)}) = \left(\frac{\partial S}{\partial x_1}(\vec{x}^{(k)}), \frac{\partial S}{\partial x_2}(\vec{x}^{(k)}), \dots, \frac{\partial S}{\partial x_n}(\vec{x}^{(k)}) \right)$,

y $\vec{x}^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})$.

→ En principio el procedimiento nos debe llevar al mínimo de S por el camino más rápido.



→ ¿Cómo elegimos el valor de α ?

- Si α es demasiado pequeño, necesitaremos un número grande de pasos.
- Si α es demasiado grande, nos podemos pasar de largo.

→ Deberíamos elegir α de manera que minimice el valor de la función $S(\alpha)$ = $S(\bar{x}^{(k+1)}) = S(\bar{x}^{(k)} - \alpha \nabla S(\bar{x}^{(k)}))$ en la dirección negativa del gradiente, pero este problema es complicado.

→ En la práctica, si suponemos que la función $S(\alpha)$ es una función cuadrática en α , se calcula la función $S(\alpha)$ con tres valores distintos de α , α_1 , α_2 y α_3 , que esperamos estén cerca de donde ocurre el mínimo de $S(\alpha)$.

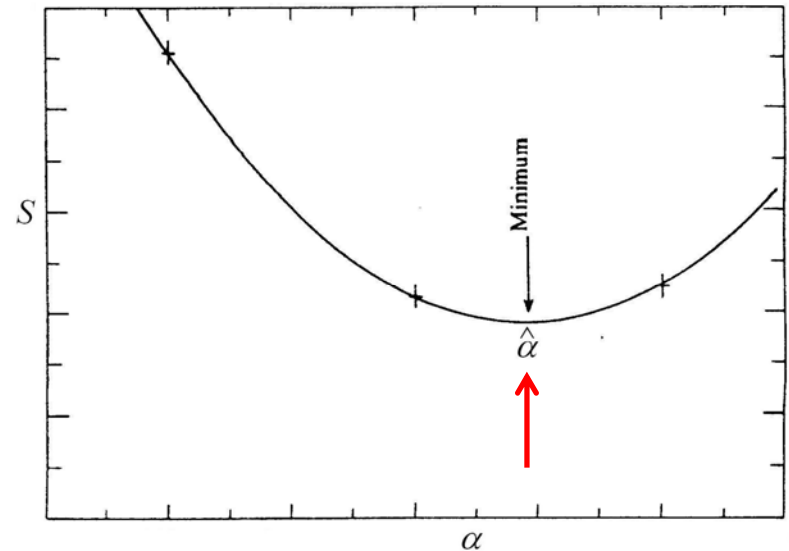
→ Estos tres puntos $(\alpha_1, S(\alpha_1))$, $(\alpha_2, S(\alpha_2))$ y $(\alpha_3, S(\alpha_3))$ se pueden ajustar a una parábola con objeto de encontrar el mínimo de la misma, $\hat{\alpha}$, que nos daría el valor óptimo de α que deberíamos utilizar en la relación de recurrencia inicial: $\bar{x}^{(k+1)} = \bar{x}^{(k)} - \hat{\alpha} \nabla S(\bar{x}^{(k)})$.



Procedimiento: para hallar el valor óptimo de α y encontrar el mínimo de $S(x_1, x_2, \dots, x_n)$.

- (1) - Seleccionar valores de partida para $\bar{x}^{(k)}$ y α , y calcular $S(\alpha) = S(\bar{x}^{(k)} - \alpha \nabla S(\bar{x}^{(k)}))$.
- (2) - Incrementar α por $\pm \Delta\alpha$ y calcular de nuevo $S(\alpha)$. El signo de $\Delta\alpha$ se elige de manera que $S(\alpha)$ disminuya.
- (3) - Repetir (2) hasta que $S(\alpha)$ pare de disminuir y comience a aumentar. Esto indica que se ha atravesado el mínimo con respecto a α .
- (4) - Usar los tres últimos valores de α cerca del mínimo y sus valores de $S(\alpha)$ para determinar el mínimo de la parábola que pasa por tres puntos, $\hat{\alpha}$. $\rightarrow \bar{x}^{(k+1)} = \bar{x}^{(k)} - \hat{\alpha} \nabla S(\bar{x}^{(k)})$

- (5) - El valor calculado para $\bar{x}^{(k+1)}$ al final del paso (4) se toma como nuevo valor de partida y se repiten los pasos (2) a (4) hasta que $S(x_1, x_2, \dots, x_n) \approx 0$.



$\rightarrow$ Alternativa más eficiente : Método del gradiente conjugado (ver Burden/Faires).

Ejemplo del paso de una función como parámetro de otra función:

```
#include <iostream>
#include <cmath>
using namespace std;
double f(double x);
double derivada(double (*f)(double), double x);
// calcula la derivada de f=sin(x) en 400 puntos equidistantes dentro del intervalo [0,2]
int main(){
double pi=acos(-1.0), x, delta=(2.0*pi)/400;
x=delta/2.0;
while(x<2*pi){
    cout << x << " " << derivada(f,x) << endl; // en la llamada, sólo se pasa el nombre de la función, f.
    x=x+delta; // La función f pasada a la función derivada tiene que ser compatible con la función
               // declarada como argumento en la función derivada.
}
return 0;
}

double f(double x) {
    return sin(x);}

double derivada(double (*f) (double), double x)
{
    // como el prototipo de la función, pero con el nombre de la función entre ( ) y precedido de *
    double h=0.0001*x, aux; // x distinto de cero
    aux=(f(x+h)-f(x))/h;
    return aux;
}
```

Ejemplo del paso de una función como parámetro de otra función:

// Cálculo de la suma de los n primeros terminos de una serie genérica.

#include <iostream>

using namespace std;

double Sumaterminos (int n, double (*funcion) (int));

double terminoserie1 (int i);

double terminoserie2 (int i);

int main()

{

cout << "<Suma de cinco terminos de la serie1: " << Sumaterminos(5, terminoserie1) << endl;

cout << "<Suma de cuatro terminos de la serie2: " << Sumaterminos(4, terminoserie2) << endl;

// en la llamada, sólo se pasa el nombre de la función, terminoserie1 o terminoserie2.

return 0;

}

double Sumaterminos (int n, double (*funcion) (int))

//argumentos: número de términos a sumar y una función que describe el término genérico de la serie

{

double suma =0;

for (int i = 1; i <= n; i++)

suma += funcion(i);

return suma;

}

double terminoserie1 (int i)

{ return (double)3.0/(i*i); }

double terminoserie2 (int i)

{ return (double)1.0/i; }

Ejemplo de creación de un array de punteros a funciones:

// Este programa evalúa las funciones f1, f2 y f3 utilizando un array de punteros a función

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
float f1 (float x);
```

```
float f2 (float x);
```

```
float f3 (float x);
```

```
const int n=3;
```

```
float minx=0.3, maxx=4.1, incremento=0.5;
```

```
int main()
```

```
{  
    float (*Af[n]) (float)={f1,f2,f3}; // se inicializa el array con n funciones
```

```
    // Af, es un array de punteros a funciones que retornan reales
```

```
    for (int i = 0; i < n; i++) {
```

```
        cout << " funcion " << i + 1 << endl;
```

```
        for (float x = minx; x < maxx; x += incremento)
```

```
            cout << " x = " << x << " f = " << Af[i](x) << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

```
float f1 (float x)
```

```
{ return (3*sin(x)+2.5*cos(x)); }
```

```
float f2 (float x)
```

```
{ return (-x*sin(x)+x*x); }
```

```
float f3 (float x)
```

```
{ return (x*x - x + 1); }
```

Algoritmo de Newton-Raphson para la resolución de sistemas de ecuaciones no lineales:

// Forma 1: no aproximamos numericamente la derivada

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
const int n=2; // numero de ecuaciones del sistema
```

```
const double PI=4.0*atan(1.0), e=exp(1.0);
```

```
int contador=0;
```

```
double tolerancia=1e-8;
```

```
void lu(double A[n][n], double b[n], double x[n], int n);
```

```
double f1 (double x[n]), f2 (double x[n]);
```

```
double f1x (double x[n]), f1y (double x[n]);
```

```
double f2x (double x[n]), f2y (double x[n]);
```

```
double norma (double x[n], int n);
```

```
int main(){
```

```
    double J[n][n], F[n], dx[n];
```

```
    double xold[n]={0.6,3.0}, xnew[n]={0.6,3.0}; // estimaciones iniciales de partida
```

```
    double (*Af[n]) (double x[n])={f1, f2}; // arrays de punteros a funciones que retornan reales
```

```
    double (*AfD[n][n]) (double x[n])={f1x, f1y, f2x, f2y}; // array de punteros a funciones nxn
```

```

do {
    for(int i=0;i<=n-1;i++){
        xold[i]=xnew[i]; }
    for (int i=0; i<=n-1; i++) {    // construimos F
        F[i]= - Af[i](xold); }
    for (int i=0; i<=n-1; i++) { // construimos la matriz jacobiana, J
        for (int j=0; j<=n-1; j++) {
            J[i][j]= AfD[i][j] (xold);
        }
    }
    lu(J,F,dx,n); // resolvemos el sistema  $J \cdot dx = F$ , llamando a la función lu
    contador=contador+1; // número de iteraciones
    cout << "Iteracion " << contador << endl;
    for(int i=0;i<=n-1;i++){
        xnew[i]=xold[i]+dx[i]; // estimación mejorada de la raíz
        cout<< xnew[i]<<" "; }
    cout << endl;}
while (fabs(norma(xnew,n)-norma(xold,n)) > tolerancia); // criterio para detener el proceso iterativo
return 0;
}

```

double f1 (double x[n])

{ return (0.5*sin(x[0]*x[1]) - (x[1]/(4.0*PI)) - (x[0]/2.0)); } // $x \rightarrow x[0]$, $y \rightarrow x[1]$

double f2 (double x[n])

{ return ((1.0-(1.0/(4.0*PI)))*(exp(2*x[0])-e) + (e*x[1]/PI) - 2*e*x[0]); }

double f1x (double x[n])

{ return (-0.5 + (x[1]*cos(x[0]*x[1])/2.0)); }

double f1y (double x[n])

{ return (-(1.0/(4.0*PI)) + (x[0]*cos(x[0]*x[1])/2.0)); }

double f2x (double x[n])

{ return ((2.0-(1.0/(2.0*PI)))*(exp(2*x[0])) - 2*e); }

double f2y (double x[n])

{ return (e/PI); }

double norma (double x[n], int n) {

double suma=0.0;

for (int i=0; i<=n-1; i++)

{ suma=suma+x[i]*x[i]; } // $\rightarrow$

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2}$$

return (sqrt(suma));

}

void lu(double A[n][n],double b[n],double x[n], int n)

{

..... } // codigo de la práctica Método LU

Algoritmo de Newton-Raphson para la resolución de sistemas de ecuaciones no lineales:

// **Forma 2:** aproximamos numericamente las derivadas parciales

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
const int n=2; // numero de ecuaciones del sistema
```

```
const double PI=4.0*atan(1.0), e=exp(1.0);
```

```
int contador=0;
```

```
double tolerancia=1e-8;
```

```
void lu(double A[n][n], double b[n], double x[n], int n);
```

```
double f1 (double x[n]);
```

```
double f2 (double x[n]);
```

```
double derivada(double (*funcion)(double x[n]), int j, double x[n]); ←
```

```
double norma (double x[n], int n);
```

```
int main(){
```

```
    double J[n][n], F[n], dx[n];
```

```
    double xold[n]={0.6,3.0}, xnew[n]={0.6,3.0}; // estimaciones iniciales de partida
```

```
    double (*Af[n]) (double x[n])={f1, f2}; // array de punteros a funciones que retornan reales
```

```

do {
    for(int i=0;i<=n-1;i++){
        xold[i]=xnew[i]; }
    for (int i=0; i<=n-1; i++) {    // construimos F
        F[i]= - Af[i](xold); }
    for (int i=0; i<=n-1; i++) { // construimos la matriz jacobiana, J
        for (int j=0; j<=n-1; j++) {
            J[i][j]=derivada(Af[i], j, xold); 
        }
    }
    lu(J,F,dx,n); // resolvemos el sistema  $J \cdot dx = F$ , llamando a la funcion lu
    contador=contador+1; // número de iteraciones
    cout << "Iteracion " << contador << endl;
    for(int i=0;i<=n-1;i++){
        xnew[i]=xold[i]+dx[i]; // estimación mejorada de la raíz
        cout<< xnew[i]<<" "; }
    cout << endl;}
while (fabs(norma(xnew,n)-norma(xold,n)) > tolerancia); // criterio para detener el proceso iterativo
return 0;
}

```

```
double f1 (double x[n])
```

```
{ return (0.5*sin(x[0]*x[1]) - (x[1]/(4.0*PI)) - (x[0]/2.0)); } // x → x[0], y → x[1]
```

```
double f2 (double x[n])
```

```
{ return ((1.0-(1.0/(4.0*PI)))*(exp(2*x[0])-e) + (e*x[1]/PI) - 2*e*x[0]); }
```

```
double derivada(double (*f)(double x[n]), int j, double x[n]) {
```

```
    double h[n], xh[n], cociente;
```

```
    for (int k=0; k<=n-1; k++){
```

```
        if (k==j) {h[k]=0.0001*x[k];} // cuidado si x[k]=0
```

```
        else {h[k]=0.0;}
```

```
        xh[k]=x[k]+h[k];
```

```
    }
```

```
    cociente=(f(xh)-f(x))/h[j]; // →
```

$$\frac{\partial f_i(\vec{x}^{(k)})}{\partial x_j} \approx \frac{f_i(\vec{x}^{(k)} + \vec{e}_j h) - f_i(\vec{x}^{(k)})}{h}$$

```
    return cociente;
```

```
}
```

```
double norma (double x[n], int n) {
```

```
    double suma=0.0;
```

```
    for (int i=0; i<=n-1; i++)
```

```
        { suma=suma+x[i]*x[i]; } // →
```

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2}$$

```
    return (sqrt(suma));
```

```
}
```

```
void lu(double A[n][n],double b[n],double x[n], int n)
```

```
{ ..... } // codigo de la práctica Método LU
```